

Atomic Refs Bounded to Memory Orderings & Atomic Accessors

Document #: P2689R1
Date: 2023-01-13
Project: Programming Language C++
SG1 & LEWG
Reply-to: Christian Trott
<crtrott@sandia.gov>
Damien Lebrun-Grandie
<lebrungrandt@ornl.gov>
Mark Hoemmen
<mhoemmen@nvidia.com>
Daniel Sunderland
<dansunderland@gmail.com>
Nevin Liber
<nliber@anl.gov>

Contents

- 1 Revision History
 - 1.1 P2689R1 2023-01 (pre-Issaquah 2023) mailing
 - 1.2 Initial Version 2022-10 Mailing
 - 1.2.1 Kona 2022 SG1 polls
- 2 Rational
- 3 Design decisions
- 4 Open questions
- 5 Wording
 - 5.1 Bounded atomic ref
 - 5.1.1 Add the following just before `[atomics.types.generic]`:
 - 5.1.2 In `[version.syn]`:
 - 5.2 Atomic Accessors
 - 5.2.1 Put the following before `[mdspan.mdspan]`:
 - 5.3 In `[version.syn]`
- 6 Acknowledgments

§ 1 Revision History

§ 1.1 P2689R1 2023-01 (pre-Issaquah 2023) mailing

- Added atomic-ref-bounded exposition-only template for an atomic_ref like type bounded to a particular memory_order
- Added atomic_ref_relaxed, atomic_ref_acq_rel and atomic_ref_seq_cst alias templates
- Added basic-atomic-accessor exposition-only template
- Added atomic_accessor_relaxed, atomic_accessor_acq_rel and atomic_accessor_seq_cst templates

§ 1.2 Initial Version 2022-10 Mailing

§ 1.2.1 Kona 2022 SG1 polls

§ 1.2.1.1 SG1 Poll #1

We like the bound memory_order versions of atomic_ref (This says nothing about whether these are aliases of general a template that takes a memory_order argument.)

atomic_ref_relaxed
 atomic_ref_acq_rel (acquire for load, release for store)
 atomic_ref_seq_cst

SF	F	N	A	SA
1	11	0	1	0

Consensus

§ 1.2.1.2 SG1 Poll #2

- X) The meaning of the bound memory_order is that is the default order
 Y) The meaning of the bound memory_order is that is the only order

SX	X	N	Y	SY
1	1	3	4	2

Consensus for Y

SX : we have a variant of this type in Folly and it has been useful to be able to override this
 SY : it's weird for atomic_ref_relaxed to have a seq_cst atomic done to it; since you can re-form the _ref you can still do it

§ 1.2.1.3 SG1 Poll #3

the next version of this proposal, with these changes, should target SG1 & LEWG

atomic_accessor -> atomic_ref
 atomic_accessor_relaxed -> atomic_ref_relaxed
 atomic_accessor_acq_rel -> atomic_ref_acq_rel
 atomic_accessor_seq_cst -> atomic_ref_seq_cst

(Return to SG1 for final wording when this is in LWG.)

SF	F	N	A	SA
6	6	2	0	0

§ 2 Rational

This proposal adds three `atomic_ref` like types each bounded to a particular `memory_order`. The API differs from `atomic_ref` in that the `memory_order` cannot be specified at run time; i.e., none of its member functions take a `memory_order` parameter. In the specific case of `atomic_ref_acq_rel`, loads are done via `memory_order_acquire` and stores are done via `memory_order_release`.

Atomic Ref	memory_order	Loads	Stores
<code>atomic_ref_relaxed</code>	<code>memory_order_relaxed</code>	<code>memory_order_relaxed</code>	<code>memory_order_relaxed</code>
<code>atomic_ref_acq_rel</code>	<code>memory_order_acq_rel</code>	<code>memory_order_acquire</code>	<code>memory_order_release</code>
<code>atomic_ref_seq_cst</code>	<code>memory_order_seq_cst</code>	<code>memory_order_seq_cst</code>	<code>memory_order_seq_cst</code>

This proposal also adds four atomic accessors to be used with `mdspan`. They only differ with their reference types and all operations are performed atomically by using that corresponding reference type.

Accessor	reference
<code>atomic_accessor</code>	<code>atomic_ref</code>
<code>atomic_accessor_relaxed</code>	<code>atomic_ref_relaxed</code>
<code>atomic_accessor_acq_rel</code>	<code>atomic_ref_acq_rel</code>
<code>atomic_accessor_seq_cst</code>	<code>atomic_ref_seq_cst</code>

`atomic_accessor` was part of the rational provided in P0009 for `mdspans` *accessor policy* template parameter.

One of the primary use cases for these accessors is the ability to write algorithms with somewhat generic `mdspan` outputs, which can be called in sequential and parallel contexts. When called in parallel contexts users would simply pass an `mdspan` with an atomic accessor - the algorithm implementation itself could be agnostic to the calling context.

A variation on this use case is an implementation of an algorithm taking an execution policy, which adds the atomic accessor to its output argument if called with a parallel policy, while using the `default_accessor` when called with a sequential policy. The following demonstrates this with a function computing a histogram:

```
template<class T, class Extents, class LayoutPolicy>
auto add_atomic_accessor_if_needed(
    std::execution::sequenced_policy,
    mdspan<T, Extents, LayoutPolicy> m) {
    return m;
}

template<class ExecutionPolicy, class T, class Extents, class LayoutPolicy>
auto add_atomic_accessor_if_needed(
    ExecutionPolicy,
    mdspan<T, Extents, LayoutPolicy> m) {
    return mdspan(m.data_handle(), m.mapping(), atomic_accessor<T>());
}
```

```

template<class ExecT>
void compute_histogram(ExecT exec, float bin_size,
                      mdspan<int, stdex::dextents<int,1>> output,
                      mdspan<float, stdex::dextents<int,1>> data) {

    static_assert(is_execution_policy_v<ExecT>);

    auto accumulator = add_atomic_accessor_if_needed(exec, output);

    for_each(exec, data.data_handle(), data.data_handle()+data.extent(0), [=](float
        val) {
        int bin = std::abs(val)/bin_size;
        if(bin > output.extent(0)) bin = output.extent(0)-1;
        accumulator[bin]++;
    });
}

```

The above example is available on godbolt: <https://godbolt.org/z/jY17Yoje1>.

§ 3 Design decisions

Three options for atomic refs were discussed in SG1 in Kona 2022: add new types for `memory_order` bounded atomic refs, add a new `memory_order` template parameter to the existing `atomic_ref`, or add a constructor to the existing `atomic_ref` that takes a `memory_order` and stores it. Given that the last two are ABI breaks, the first option was polled and chosen. It was also decided that the new bounded atomic refs would not support overriding the specified `memory_order` at run time.

This proposal has chosen to make a general exposition-only template `atomic-ref-bounded` that takes a `memory_order` as a template argument and alias templates for the three specific bounded atomic refs. Also, the various member functions are constrained by integral types not including `bool`, floating point types and pointer types, as opposed to the different template specializations specified for `atomic_ref`. Other than not being able to specify the `memory_order` at run time, the intention is that the bounded atomic ref types have the same functionality and API as `atomic_ref`.

Similarly for the atomic accessors, it was decided in SG1 in Kona 2022 to add four new types. This proposal has chosen to make a general exposition-only template `basic-atomic-accessor` which takes the reference type as a template parameter, and four alias templates for the specific atomic accessors.

§ 4 Open questions

This proposal uses alias templates to exposition-only types for `atomic_ref_relaxed`, `atomic_accessor`, etc. However, we do not want to prescribe a particular implementation. For instance, if an implementer wished to derive from an `atomic-ref-bounded-like` type (to get a more user friendly name mangling, which is something not normally covered by the Standard), would they be allowed to do so? We believe an alias template to an exposition-only type is not observable from the point of view of the Standard and such an implementation would be allowed, but we request clarification on this.

§ 5 Wording

The proposed changes are relative to [N4917](#):

§ 5.1 Bounded atomic ref

§ 5.1.1 Add the following just before [atomics.types.generic]:

Class template atomic-ref-bounded [atomics.ref.bounded]

General [atomics.ref.bounded.general]

```
// all freestanding
template <class T, memory_order MemoryOrder>
struct atomic-ref-bounded { // exposition only
private:
    using atomic_ref_unbounded = atomic_ref<T>; // exposition only
    atomic_ref_unbounded ref; // exposition only

    static constexpr memory_order store_ordering =
        MemoryOrder == memory_order_acq_rel ? memory_order_release
        : MemoryOrder; // exposition only

    static constexpr memory_order load_ordering =
        MemoryOrder == memory_order_acq_rel ? memory_order_acquire
        : MemoryOrder; // exposition only

    static constexpr bool is_integral_value =
        is_integral_v<T> && !is_same_v<T, bool>; // exposition only
    static constexpr bool is_floating_point_value =
        is_floating_point_v<T>; // exposition only
    static constexpr bool is_pointer_value =
        is_pointer_v<T>; // exposition only

public:
    using value_type = T;
    using difference_type = ptrdiff_t;
    static constexpr memory_order memory_ordering = MemoryOrder;
    static constexpr size_t required_alignment =
        atomic_ref_unbounded::required_alignment;

    static constexpr bool is_always_lock_free =
        atomic_ref_unbounded::is_always_lock_free;
    bool is_lock_free() const noexcept;

    explicit atomic-ref-bounded(T& t);
    atomic-ref-bounded(atomic-ref-bounded const& a) noexcept;
    atomic-ref-bounded& operator=(atomic-ref-bounded const&) = delete;

    void store(T desired) const noexcept;
    T operator=(T desired) const noexcept;
    T load() const noexcept;
    operator T() const noexcept;

    T exchange(T desired) const noexcept;
    bool compare_exchange_weak(T& expected, T desired) const noexcept;
    bool compare_exchange_strong(T& expected, T desired) const noexcept;

    T fetch_add(T operand) const noexcept;
    T fetch_add(difference_type operand) const noexcept;
    T fetch_sub(T operand) const noexcept;
    T fetch_sub(difference_type operand) const noexcept;

    T fetch_and(T operand) const noexcept;
    T fetch_or(T operand) const noexcept;
```

```

    T fetch_xor(T operand) const noexcept;

    T operator++(int) const noexcept;
    T operator++() const noexcept;
    T operator--(int) const noexcept;
    T operator--() const noexcept;

    T operator+=(T operand) const noexcept;
    T operator+=(difference_type operand) const noexcept;
    T operator-=(T operand) const noexcept;
    T operator-=(difference_type operand) const noexcept;

    T operator&=(T operand) const noexcept;
    T operator|=(T operand) const noexcept;
    T operator^=(T operand) const noexcept;

    void wait(T old) const noexcept;
    void notify_one() const noexcept;
    void notify_all() const noexcept;
};

```

1 Class `atomic-ref-bounded` is for exposition only.

2 *Mandates:*

(2.1) — `is_trivially_copyable_v<T>` is true.

(2.2) — `is_same_v<T, remove_cv_t<T>>` is true.

[Note: Unlike `atomic_ref`, the memory ordering for the arithmetic operators is `MemoryOrder`, which is not necessarily `memory_order_seq_cst`. — end note]

Operations [`atomics.ref.bounded.ops`]

```
bool is_lock_free() const noexcept;
```

1 *Effects:* Equivalent to: `return ref.is_lock_free();`

```
explicit atomic-ref-bounded(T& t);
```

2 *Preconditions:* The referenced object is aligned to `required_alignment`.

3 *Postconditions:* `ref` references the object referenced by `t`.

4 *Throws:* Nothing.

```
atomic-ref-bounded(atomic-ref-bounded const& a) noexcept;
```

5 *Postconditions:* `*this` references the object referenced by `a`.

```
void store(T desired) const noexcept;
```

6 *Effects:* Equivalent to: `ref.store(desired, store_ordering);`

```
T operator=(T desired) const noexcept;
```

7 *Effects:* Equivalent to: `store(desired); return desired;`

```
T load() const noexcept;
```

- 8 *Effects:* Equivalent to: `ref.load(load_ordering);`
- ```
operator T() const noexcept;
```
- 9 *Effects:* Equivalent to: `return load();`
- ```
T exchange(T desired) const noexcept;
```
- 10 *Effects:* Equivalent to: `return ref.exchange(desired, memory_ordering);`
- ```
bool compare_exchange_weak(T& expected, T desired) const noexcept;
```
- 11 *Effects:* Equivalent to: `return ref.compare_exchange_weak(expected, desired, memory_ordering, load_ordering);`
- ```
bool compare_exchange_strong(T& expected, T desired) const noexcept;
```
- 12 *Effects:* Equivalent to: `return ref.compare_exchange_strong(expected, desired, memory_ordering, load_ordering);`
- ```
T fetch_add(T operand) const noexcept;
```
- 13 *Constraints:* `is_integral_value || is_floating_point_value` is true.
- 14 *Effects:* Equivalent to: `return ref.fetch_add(operand, memory_ordering);`
- ```
T fetch_add(difference_type operand) const noexcept;
```
- 15 *Constraints:* `is_pointer_value` is true.
- 16 *Effects:* Equivalent to: `return ref.fetch_add(operand, memory_ordering);`
- ```
T fetch_sub(T operand) const noexcept;
```
- 17 *Constraints:* `is_integral_value || is_floating_point_value` is true.
- 18 *Effects:* Equivalent to: `return ref.fetch_sub(operand, memory_ordering);`
- ```
T fetch_sub(difference_type operand) const noexcept;
```
- 19 *Constraints:* `is_pointer_value` is true.
- 20 *Effects:* Equivalent to: `return ref.fetch_sub(operand, memory_ordering);`
- ```
T fetch_and(difference_type operand) const noexcept;
```
- 21 *Constraints:* `is_integral_value` is true.
- 22 *Effects:* Equivalent to: `return ref.fetch_and(operand, memory_ordering);`
- ```
T fetch_or(difference_type operand) const noexcept;
```
- 23 *Constraints:* `is_integral_value` is true.
- 24 *Effects:* Equivalent to: `return ref.fetch_or(operand, memory_ordering);`
- ```
T fetch_xor(difference_type operand) const noexcept;
```
- 25 *Constraints:* `is_integral_value` is true.

26 *Effects:* Equivalent to: `return ref.fetch_xor(operand, memory_ordering);`

```
T operator++(int) const noexcept;
```

27 *Constraints:* `is_integral_value || is_pointer_value` is true.

28 *Effects:* Equivalent to: `return fetch_add(1);`

```
T operator++() const noexcept;
```

29 *Constraints:* `is_integral_value || is_pointer_value` is true.

30 *Effects:* Equivalent to: `return fetch_add(1) + 1;`

```
T operator--(int) const noexcept;
```

31 *Constraints:* `is_integral_value || is_pointer_value` is true.

32 *Effects:* Equivalent to: `return fetch_sub(1);`

```
T operator--() const noexcept;
```

33 *Constraints:* `is_integral_value || is_pointer_value` is true.

34 *Effects:* Equivalent to: `return fetch_sub(1) - 1;`

```
T operator+=(T operand) const noexcept;
```

35 *Constraints:* `is_integral_value || is_floating_point_value` is true.

36 *Effects:* Equivalent to: `return fetch_add(operand) + operand;`

```
T operator+=(difference_type operand) const noexcept;
```

37 *Constraints:* `is_pointer_value` is true.

38 *Effects:* Equivalent to: `return fetch_add(operand) + operand;`

```
T operator-=(T operand) const noexcept;
```

39 *Constraints:* `is_integral_value || is_floating_point_value` is true.

40 *Effects:* Equivalent to: `return fetch_sub(operand) - operand;`

```
T operator-=(difference_type operand) const noexcept;
```

41 *Constraints:* `is_pointer_value` is true.

42 *Effects:* Equivalent to: `return fetch_sub(operand) - operand;`

```
T operator&=(T operand) const noexcept;
```

43 *Constraints:* `is_integral_value` is true.

44 *Effects:* Equivalent to: `return fetch_and(operand) & operand;`

```
T operator|=(T operand) const noexcept;
```

45 *Constraints:* `is_integral_value` is true.

46 *Effects:* Equivalent to: `return fetch_or(operand) | operand;`

```
T operator^=(T operand) const noexcept;
```

47 *Constraints:* `is_integral_value` is true.

48 *Effects:* Equivalent to: `return fetch_xor(operand) ^ operand;`

```
void wait(T old) const noexcept;
```

49 *Effects:* Equivalent to: `ref.wait(old, load_ordering);`

```
void notify_one() const noexcept;
```

50 *Effects:* Equivalent to: `ref.notify_one(); }`

```
void notify_all() const noexcept;
```

51 *Effects:* Equivalent to: `ref.notify_all(); }`

### Memory Order Specific Atomic Refs [atomics.refs.bounded.order]

```
// all freestanding
namespace std {
template<class T>
using atomic_ref_relaxed = atomic_ref_bounded<T, memory_order_relaxed>;

template<class T>
using atomic_ref_acq_rel = atomic_ref_bounded<T, memory_order_acq_rel>;

template<class T>
using atomic_ref_seq_cst = atomic_ref_bounded<T, memory_order_seq_cst>;
}
```

#### § 5.1.2 In [version.syn]:

Update the feature test macro `__cpp_lib_atomic_ref`.

## § 5.2 Atomic Accessors

### § 5.2.1 Put the following before [mdspan.mdspan]:

#### Class template basic-atomic-accessor [mdspan.accessor.basic]

##### General [mdspan.accessor.basic.overview]

```
template <class ElementType, class ReferenceType>
struct basic_atomic_accessor { // exposition only
 using offset_policy = basic_atomic_accessor;
 using element_type = ElementType;
 using reference = ReferenceType;
 using data_handle_type = ElementType*;

 constexpr basic_atomic_accessor() noexcept = default;

 template <class OtherElementType>
 constexpr basic_atomic_accessor(default_accessor<OtherElementType>) noexcept;

 template <class OtherElementType>
```

```
constexpr basic-atomic-accessor(basic-atomic-accessor<OtherElementType,
 ReferenceType>) noexcept;

constexpr reference access(data_handle_type p, size_t i) const noexcept;
constexpr data_handle_type offset(data_handle_type p, size_t i) const noexcept;
};
```

- 1 Class basic-atomic-accessor is for exposition only.
- 2 basic-atomic-accessor meets the accessor policy requirements.
- 3 ElementType is required to be a complete object type that is neither an abstract class type nor an array type.
- 4 Each specialization of basic-atomic-accessor is a trivially copyable type that models semiregular.
- 5  $[0, n)$  is an accessible range for an object p of type data\_handle\_type and an object of type basic-atomic-accessor if and only if  $[p, p+n)$  is a valid range.

#### Members [mdspan.accessor.basic.members]

```
template <class OtherElementType>
constexpr basic-atomic-accessor(default_accessor<OtherElementType>) noexcept {}

template <class OtherElementType>
constexpr basic-atomic-accessor(basic-atomic-accessor<OtherElementType,
 ReferenceType>) noexcept {}
```

- 1 *Constraints:* is\_convertible\_v<OtherElementType (\*)[], element\_type (\*)[]> is true.

```
constexpr reference access(data_handle_type p, size_t i) const noexcept;
```

- 2 *Effects:* Equivalent to return reference(p + i);

```
constexpr data_handle_type offset(data_handle_type p, size_t i) const noexcept;
```

- 3 *Effects:* Equivalent to return p[i];

#### Atomic accessors [mdspan.accessor.atomic]

```
namespace std {
template <class ElementType>
using atomic_accessor = basic-atomic-accessor<ElementType, atomic_ref<ElementType>>;

template <class ElementType>
using atomic_accessor_relaxed = basic-atomic-accessor<ElementType,
 atomic_ref_relaxed<ElementType>>;

template <class ElementType>
using atomic_accessor_acq_rel = basic-atomic-accessor<ElementType,
 atomic_ref_acq_rel<ElementType>>;

template <class ElementType>
using atomic_accessor_seq_cst = basic-atomic-accessor<ElementType,
 atomic_ref_seq_cst<ElementType>>;
}
```

## § 5.3 In [version.syn]

Add the following feature test macro:

```
#define __cpp_lib_atomic_accessors YYYYMML // also in <mdspan>
```

## § 6 Acknowledgments

Sandia National Laboratories is a multimission laboratory managed and operated by National Technology and Engineering Solutions of Sandia, LLC., a wholly owned subsidiary of Honeywell International, Inc., for the U.S. Department of Energy's National Nuclear Security Administration under Grant DE-NA-0003525.

This manuscript has been authored by UTBattelle, LLC, under Grant DE-AC05-00OR22725 with the U.S. Department of Energy (DOE).

This work was supported by Exascale Computing Project 17-SC-20-SC, a joint project of the U.S. Department of Energy's Office of Science and National Nuclear Security Administration, responsible for delivering a capable exascale ecosystem, including software, applications, and hardware technology, to support the nation's exascale computing imperative.

This research used resources of the Argonne Leadership Computing Facility, which is a DOE Office of Science User Facility supported under Contract DE-AC02-06CH11357.